Busca en espazos de estados*



Senén Barro Ameneiro, CiTIUS

@SenenBarro

*Baseado parcialmente en:
http://elvex.ugr.es/courses.html

Busca de solucións

Método	Descripción	Tipo de solución
Preciso	Método específico do problema e que aporta unha solución óptima	Exacta
Heurístico	Utilízase un método de resolución de problemas que aplica coñecemento específico do problema con obxecto de aproximar unha solución ao mesmo ou aumentar a eficiencia na procura dunha solución	Aproximada
Metaheurístico	Melloran os procedementos heurísticos xeralizándoos e obtendo melloras na eficiencia. En xeral utilizan estratexias que diversifican a busca para evitar mínimos locais, intensificándoa en zonas prometedoras	Aproximada

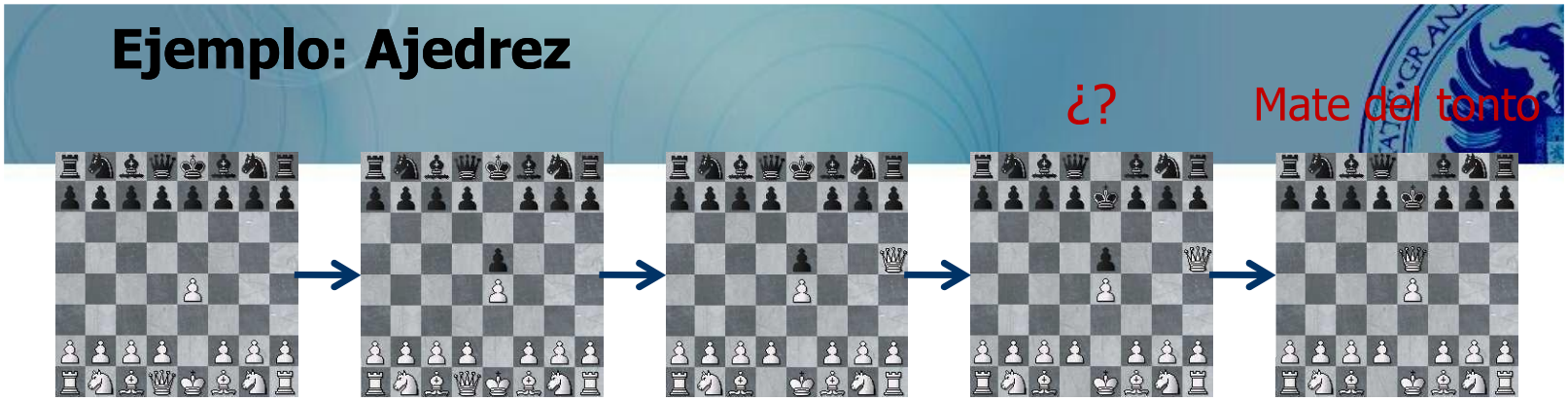
Aproximacións á resolución de problemas

Busca en espazos de estados



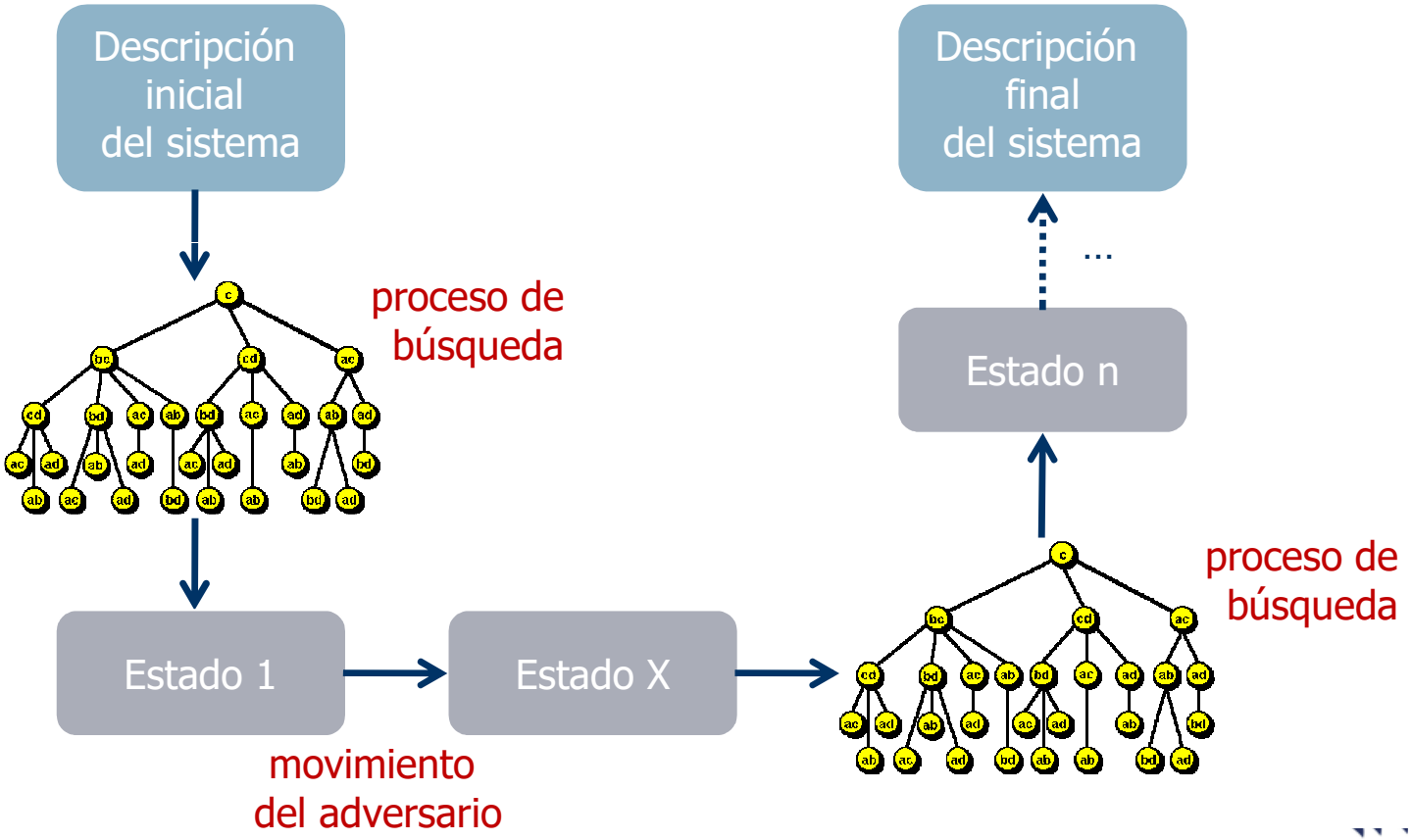
Solución: estado **n** ou secuencia de estados de **1** a **n**

Busca en espazos de estados



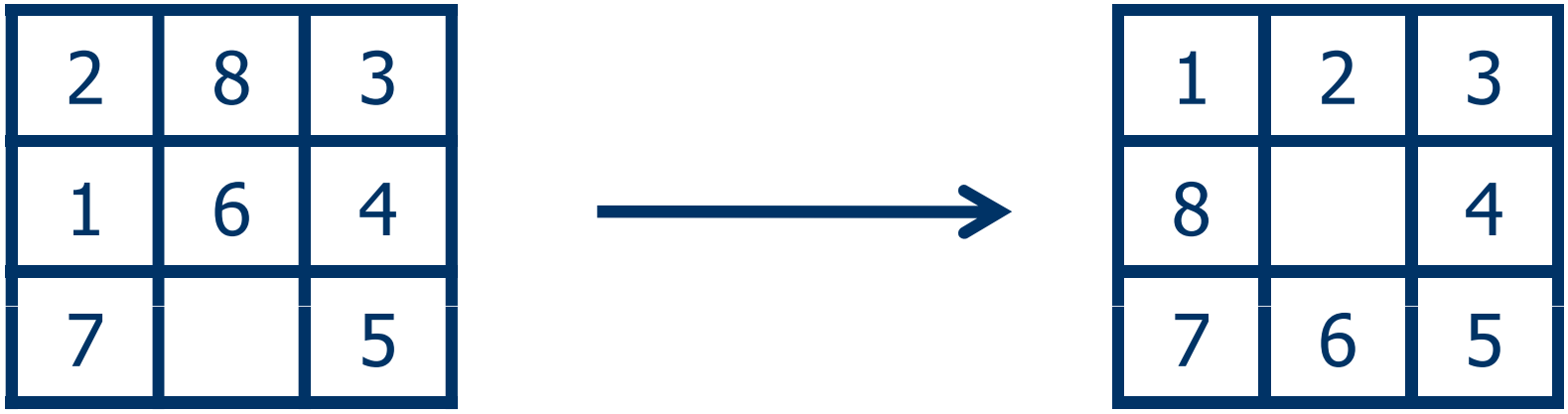
Solución: jaque mate ou, como mal menor, táboas

Busca en espazos de estados



Solución: acadar un **estado meta** (ex.: xadrez)

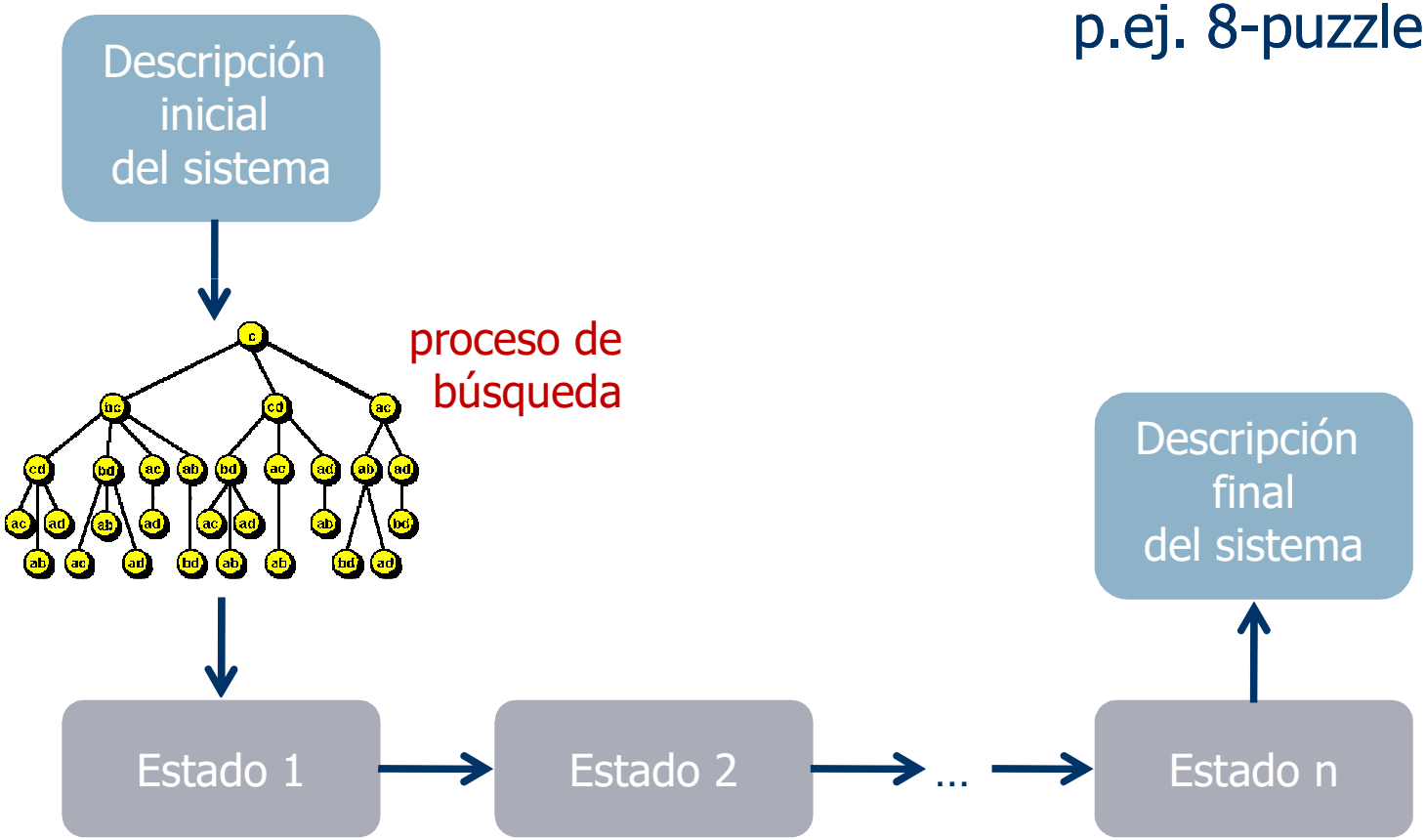
Busca en espazos de estados



Problema 8-quebracabezas

Busca en espazos de estados

p.ej. 8-puzzle



Solución: **secuencia** ata un estado meta

Busca en espazos de estados

- **Espazo de estados**: estados aos que se pode chegar desde o **estado inicial** mediante calquera secuencia de **operadores**
- O proceso de **busca** trata de atopar unha meta ou solución ao problema –a veces a **ruta** seguida-
- **Proba de meta**: permite saber se un estado dado é meta ou solución do problema
- **Condición de parada** –parcial ou total-: criterio/s para deter a busca

Busca en espazos de estados

Definición do problema 8-quebracabezas

- **Estados:** disposición das pezas
- **Operadores:** movemento do espazo baleiro ^,v,>,<
- **Proba de meta:** chegouse ao estado final?
- **Custo de ruta:** número de movementos realizados desde o estado inicial á meta

Busca en espazos de estados

```
  F O R T Y
+     T E N
+     T E N
  _ _ _ _ _
  S I X T Y
```

```
    2 9 7 8 6
+       8 5 0
+       8 5 0
  _ _ _ _ _
  3 1 4 8 6
```

criptoaritmética

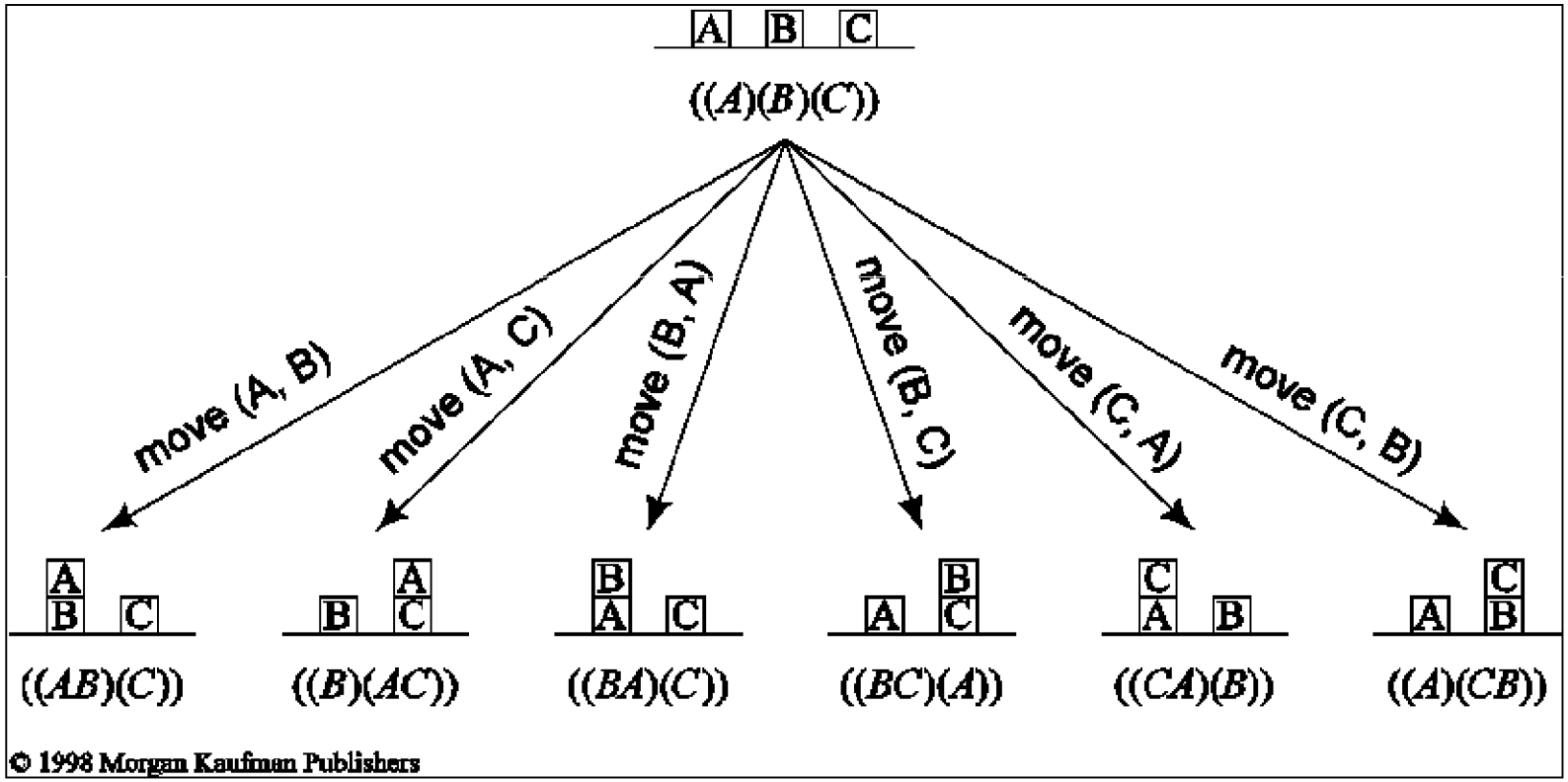
Estados: sustitución de letras por dixitos

Operadores: unha letra sustitúese por un mesmo díxito, sen repeticións

Proba de meta: só díxitos e unha suma correcta

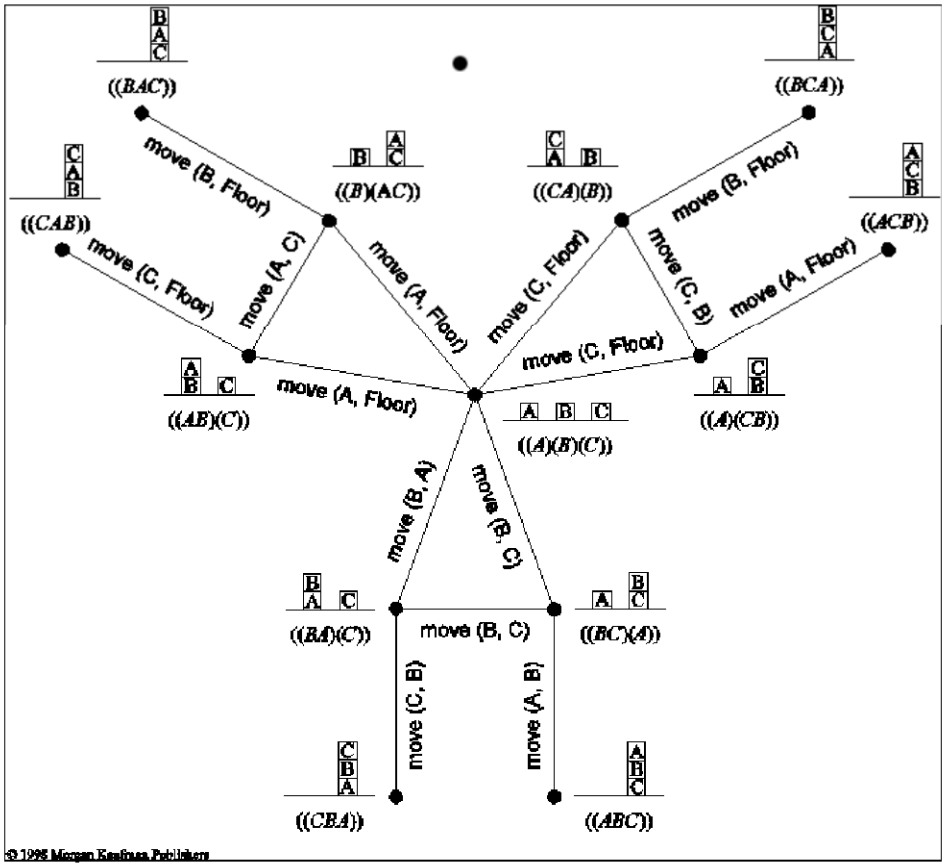
Custo de ruta: Cero

Busca en espazos de estados



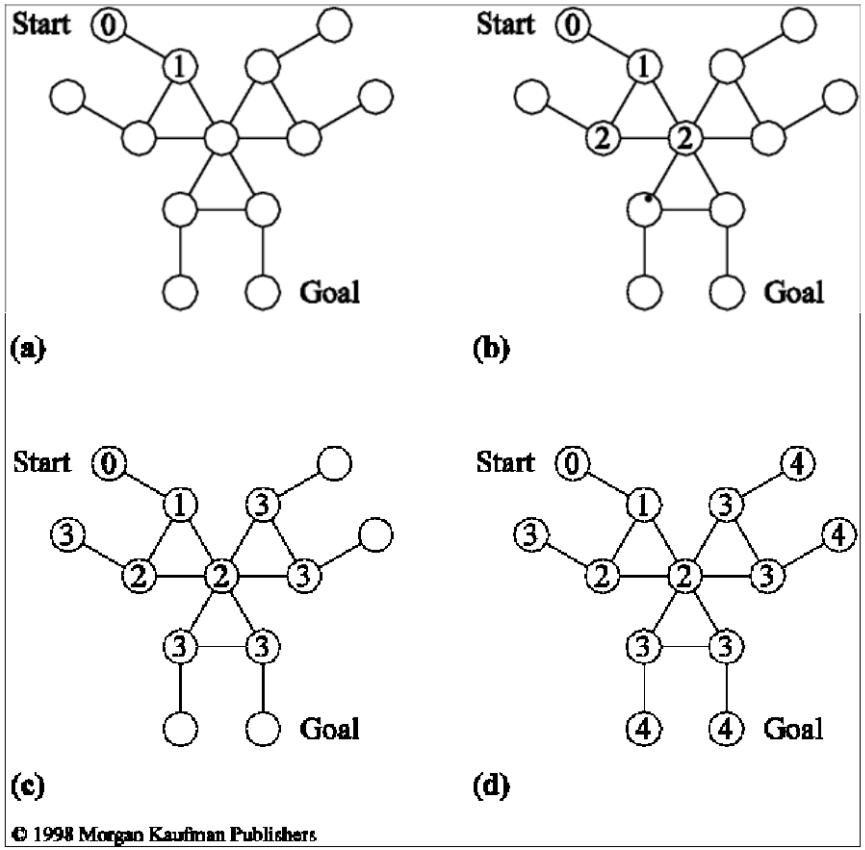
Exemplo: problema de bloques

Busca en espazos de estados



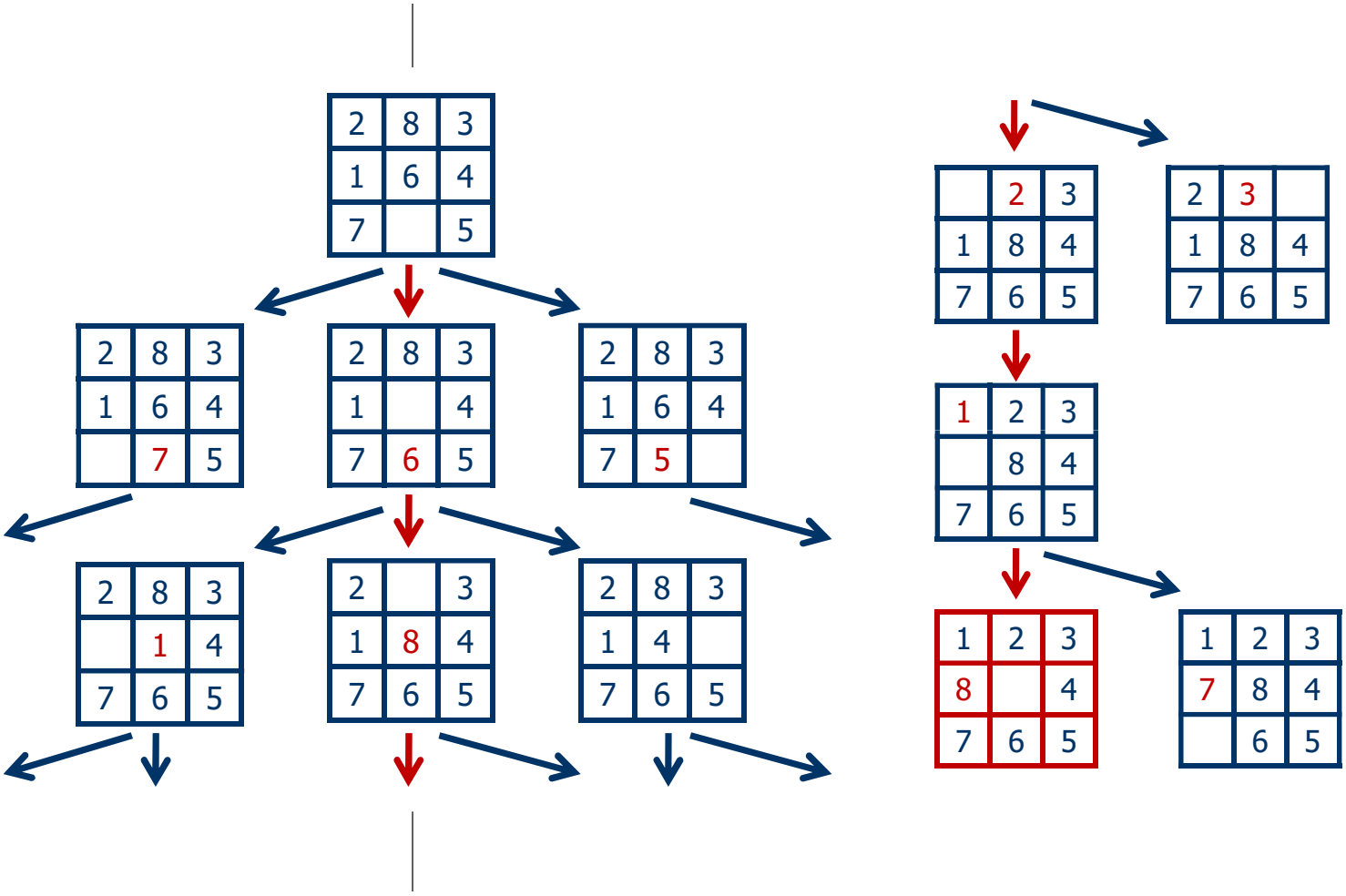
Grafo implícito nun problema de bloques

Busca en espazos de estados



Exemplo: problema de bloques

Busca en espazos de estados



Problema 8-quebracabezas

Busca en espazos de estados

- Imposibilidade de explorar todas as alternativas en **problemas complexos**
- Se existen **múltiples solucións**, normalmente é suficiente con buscar unha solución aceptable -o mellor é inimigo do bo-
- **Heurísticas**: criterios para seleccionar un operador ou acción prometedora

Busca a cegas

Función: Búsqueda en espacio de estados()

Datos: El estado inicial

Resultado: Una solución

Seleccionar el primer estado como el estado actual

mientras *estado actual* $\neq$ *estado final* **hacer**

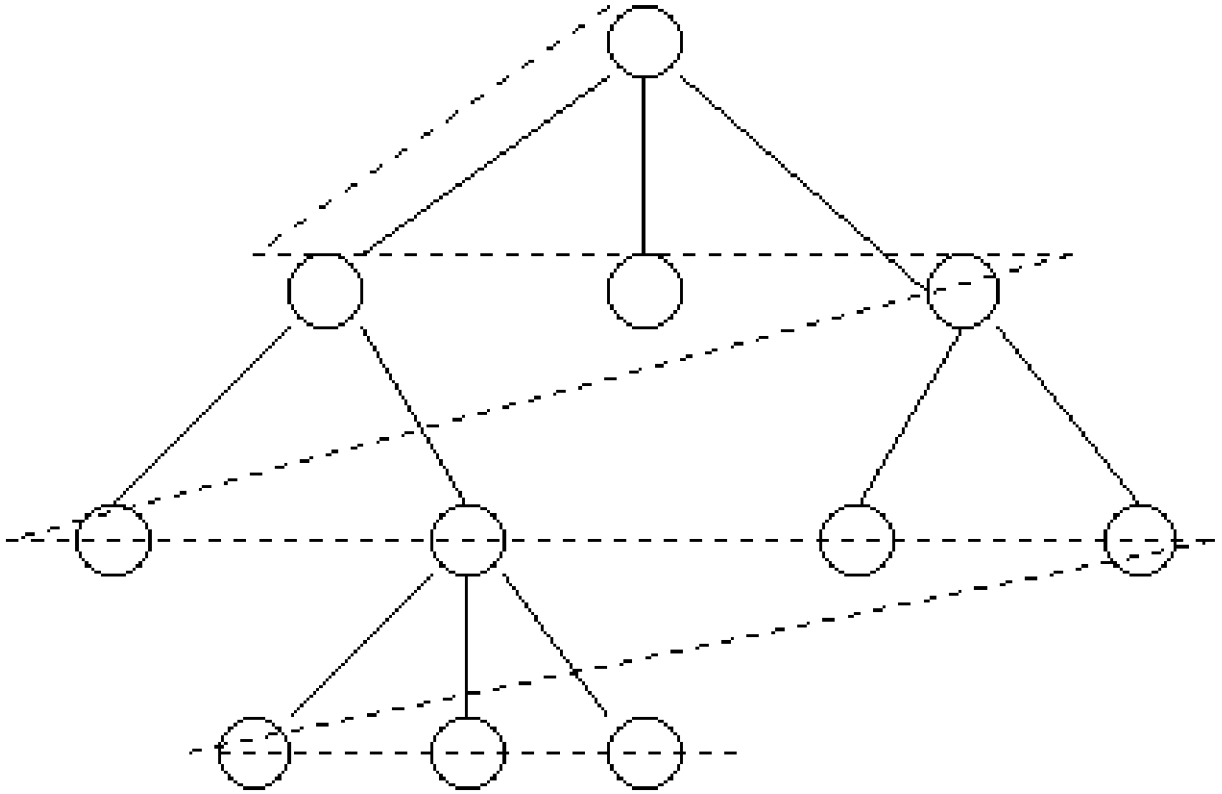
| Generar y guardar sucesores del estado actual (expansión)

| Escoger el siguiente estado entre los pendientes (selección)

fin

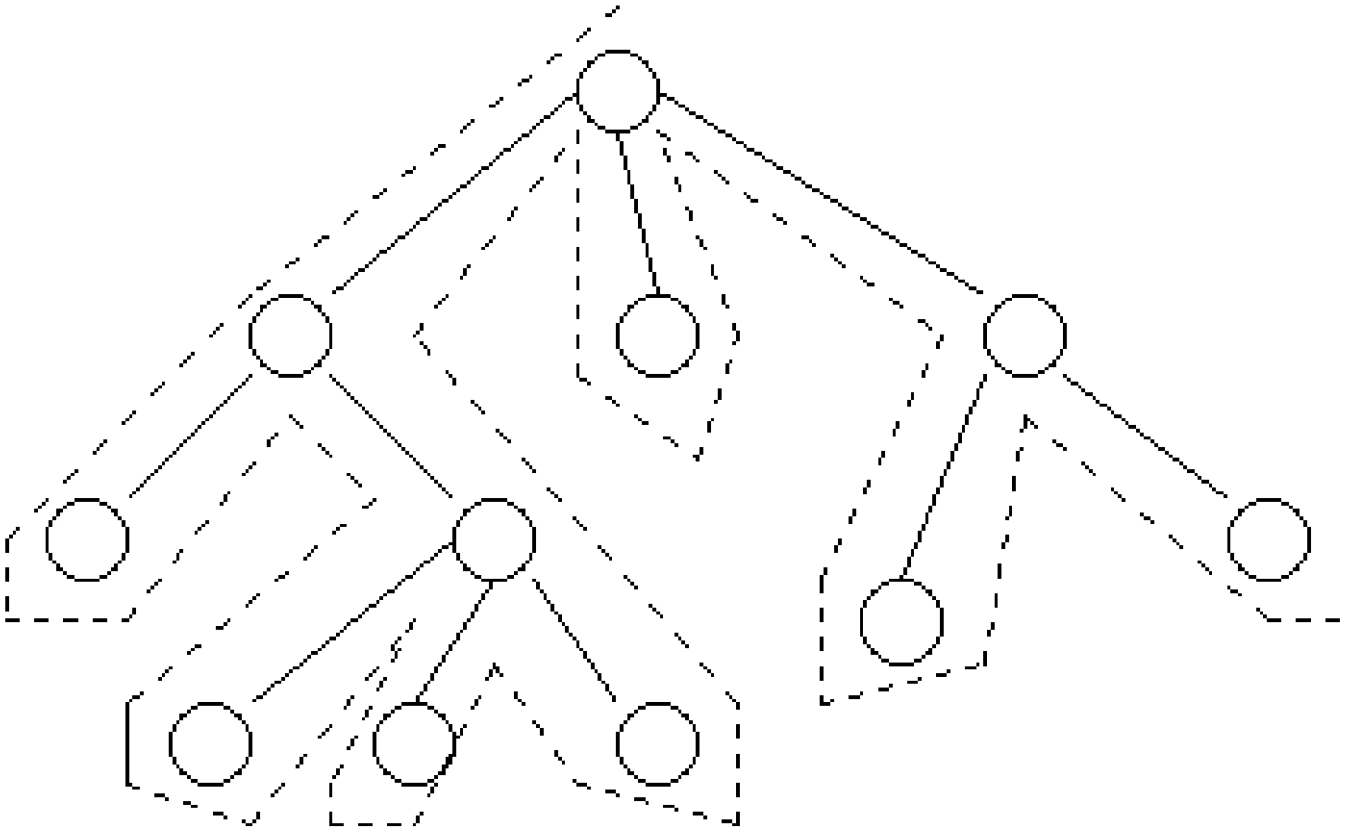
Exploración exhaustiva de estados

Busca a cegas



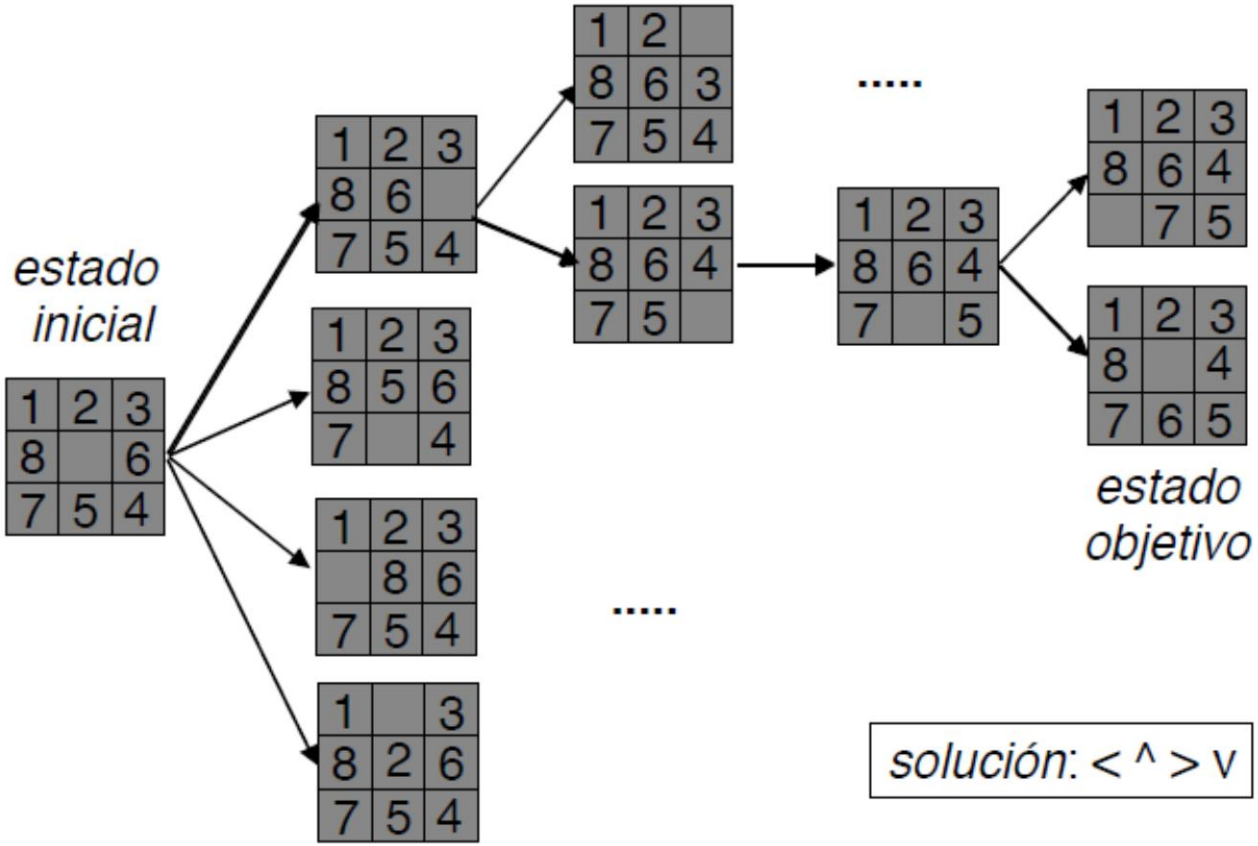
Busca en amplitude

Busca a cegas



Busca en profundidade

Busca a cegas



8-quebracabezas en profundidade

Busca a cegas

Algoritmo de busca en **amplitude**:

1. Crear lista de nós ABERTA co nó raíz
2. Ata que ABERTA esté baleira ou se atope un estado solución, realizar:
    1. Extraer o primeiro nó de ABERTA (**m**) –FIFO-
    2. Expandir **m**. Para cada operador aplicable e cada forma de aplicación:
        1. Aplicar operador a **m**, obtendo un novo estado e creando un punteiro que permita saber que o seu predecesor é **m**
        2. Se o novo estado é solución, saír e devolver dito estado
        3. Incluir o novo estado ao final de ABERTA

Busca a cegas

Algoritmo de busca en **profundidade [limitada]**:

1. Crear lista de nós ABERTA co nó raíz
2. Ata que se atope un estado solución ou un fallo, realizar:
    1. Se ABERTA está baleira, fallo. Noutro caso:
    2. Extraer o primeiro nó de ABERTA (**m**) –LIFO-
    3. Se a profundidade de **m** é igual a **p**, retroceder a 2. Noutro caso continuar
    4. Expandir **m** creando punteiros cara el desde os seus sucesores Introducir os sucesores en ABERTA:
        1. Se algún sucesor de **m** é solución, devolver camiño da solución
        2. Se algún sucesor de **m** está nun "punto morto", eliminalo de ABERTA
        3. Retroceder a (2)

Busca en espazos de estados

Avaliación das estratexias seguidas:

- Busca **completa**: se existe solución, atópaa
- Busca **óptima**: atopa a mellor solución, se existe
- **Complexidade**:
  - **Temporal**: número de nós explorados
  - **Espacial**: Máximo número de nós en memoria de forma simultánea
- **Terminoloxía**:
  - **Factor de ramificación** (**r**): número medio de sucesores dos nós
  - **Profundidade da solución** (**p**)

Busca en espazos de estados

Profundidade	Nós	Tempo	Memoria
0	1	0,001 mseg	100 Byte
2	111	0,1 mseg	11 KB
4	11.111	11 mseg	1 MB
6	10^6	1 seg	111 MB
8	10^8	100 seg	11 GB
10	10^{10}	0,128 días	1 TB
12	10^{12}	12,8 días	111 TB
14	10^{14}	3,5 anos	11 PB

Busca en amplitude, asumindo: **r**=10; 1 M nós/seg; 100 bytes/nó

Busca en espazos de estados

	Amplitude	Profundidade
Busca completa	Sí	Non, agás en espazos finitos e sen bucles
Busca óptima	Sí	Non
Complexidade temporal	$O(r^p)$	$O(r^m)$
Complexidade espacial	$O(r^p)$	O(r*m)

Avaliación de estratexias de busca a cegas

r: factor de ramificación; p: profundidade da solución;
m: profundidade máxima da árbore de busca

Busca en espazos de estados

	Preferente por amplitude	Profundi-dade	Limitada en profundi-dade	Profundi-zación iterativa	Bidireccio-nal (cando é posible)
Tempo	r^p	r^m	r^l	r^p	$r^{p/2}$
Espazo	r^p	r*m	r*l	r*p	$r^{p/2}$
Óptima?	Si	Non	Non	Si	Si
Completa?	Si	Non	Si, cando l>p	Si	Si

Avaliación de estratexias de busca a cegas

r: factor de ramificación; p: profundidade da solución; m: profundidade máxima da árbore de busca; l: límite da profundidade

Busca heurística

Busca con **información** potencialmente útil

De modo xeral podemos avaliar os nós a través dunha **función de avaliación:**

$$f(n) = g(n) + h(n)$$

g(n) é o **custo de camiño** desde o nó de inicio

h(n) é unha **función heurística** que asigna a cada nó un valor de utilidade estimada na busca dunha solución –en xeral unha distancia a un nó solución-

Busca heurística

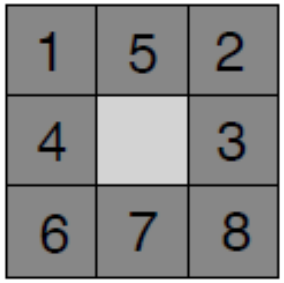
Función de avaliación:

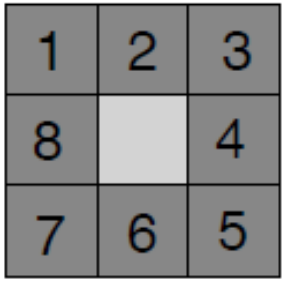
$$f(n) = g(n) + h(n)$$

- f(n) = g(n), custo uniforme; busca non informada ou **a cegas**
- f(n) = h(n), algoritmo **primeiro o mellor**
- f(n) = g(n) + h(n), algoritmo **A***

Busca heurística

estado actual





estado objetivo

Heurísticas:

piezas fuera de sitio = 7

∑ distancia Manhattan = 3 + 1 + 1 + 3 + 1 + 1 + 2 = 12
(número pasos en cuadrícula)

Heurísticas para solución do 8-quebracabezas

Busca heurística

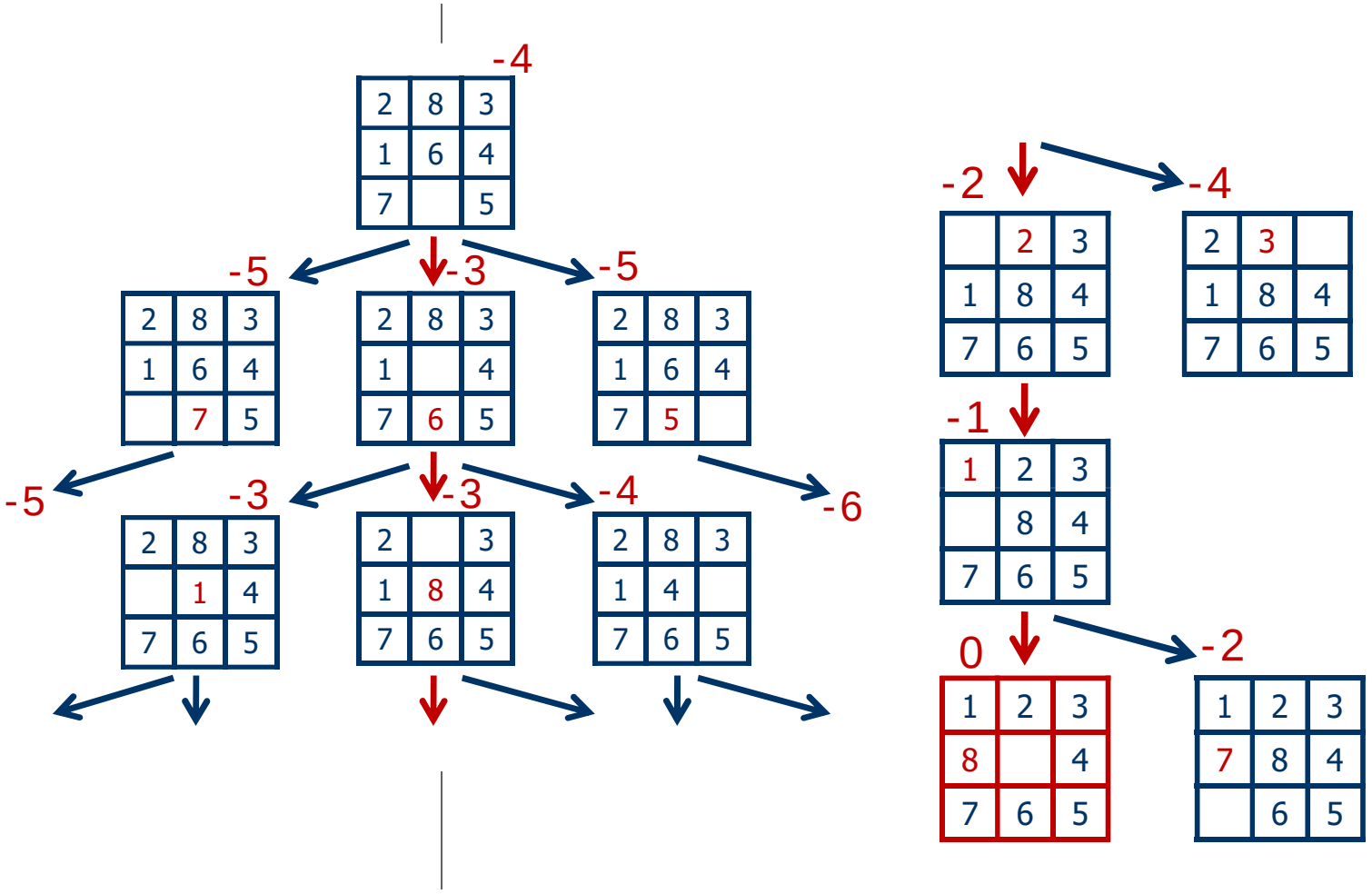


f(E) = -4

f (OBJ) = 0

Función heurística (f): número de pezas mal colocadas

Busca heurística



Función heurística (f): número de pezas mal colocadas

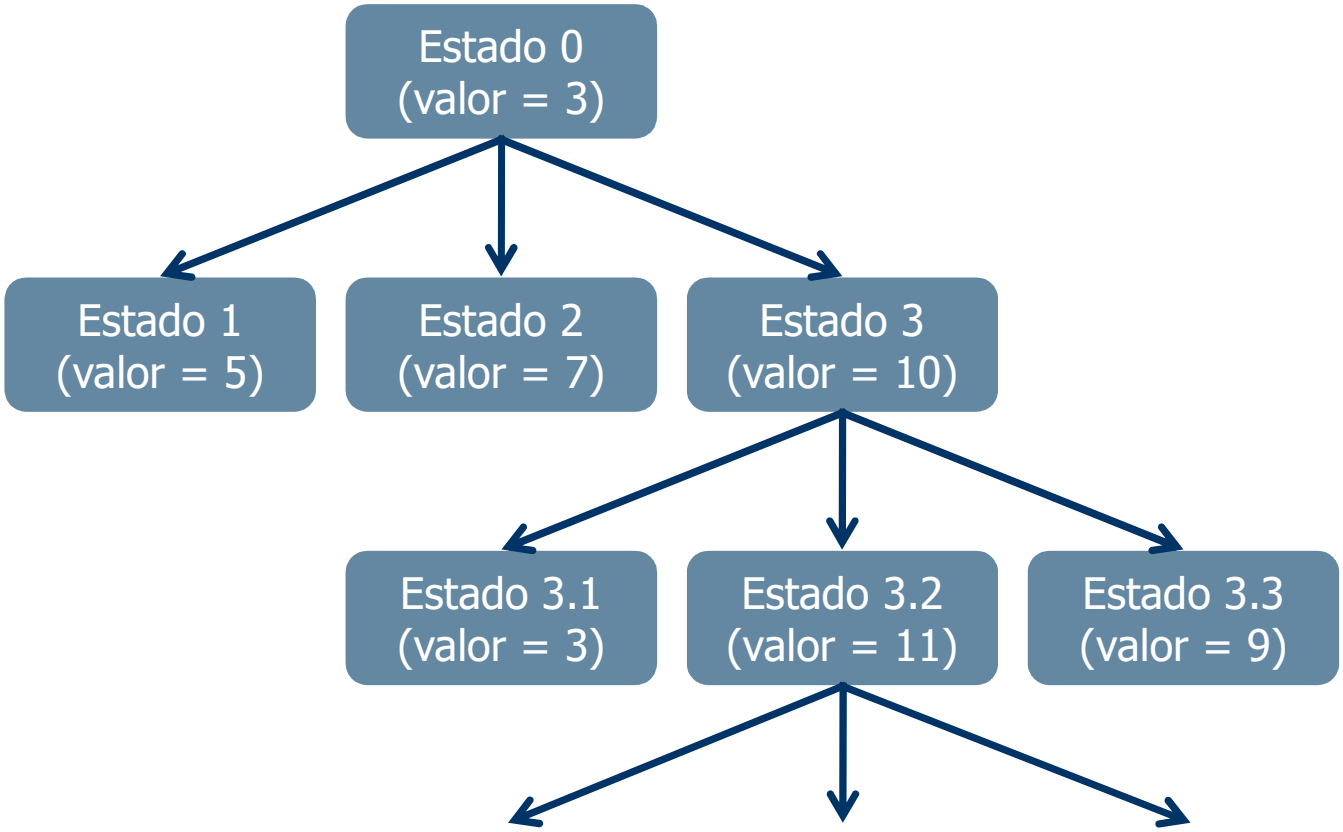
Busca primeiro o mellor

E: Estado activo

while (queden nodos por explorar a partir de E) {
  Para todos los operadores R_i, obtener $E_i=R_i(E)$
  Evaluar $f(E_i)$ para todos los estados $E_i=R_i(E)$
  Seleccionar E_{max} tal que $f(E_{max}) = max\{f(E_i)\}$
  if ($f(E_{max})>f(E)$) {
    $E = E_{max}$
  } else return E
}

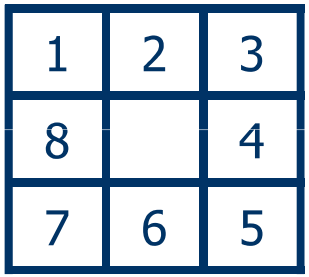
Estratexia irrevocable con información: ascensión de outeiros pola máxima pendente

Busca primeiro o mellor

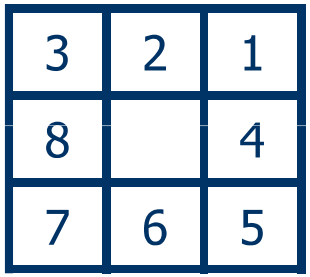


Estratexia irrevocable con información:
ascensión de outeiros pola máxima pendente

Busca primeiro o mellor



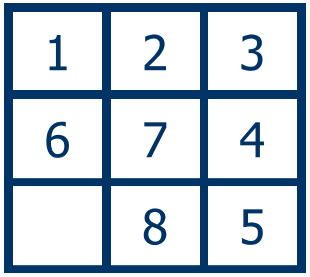
objetivo



$E_{\text{máximo local}}$

Máximo local

Todos los movimientos empeoran el valor de la función heurística.



E_{meseta}

Meseta

Todos los movimientos dejan igual el valor de la función heurística.

Ascensión por máxima pendente en 8-quebracabezas

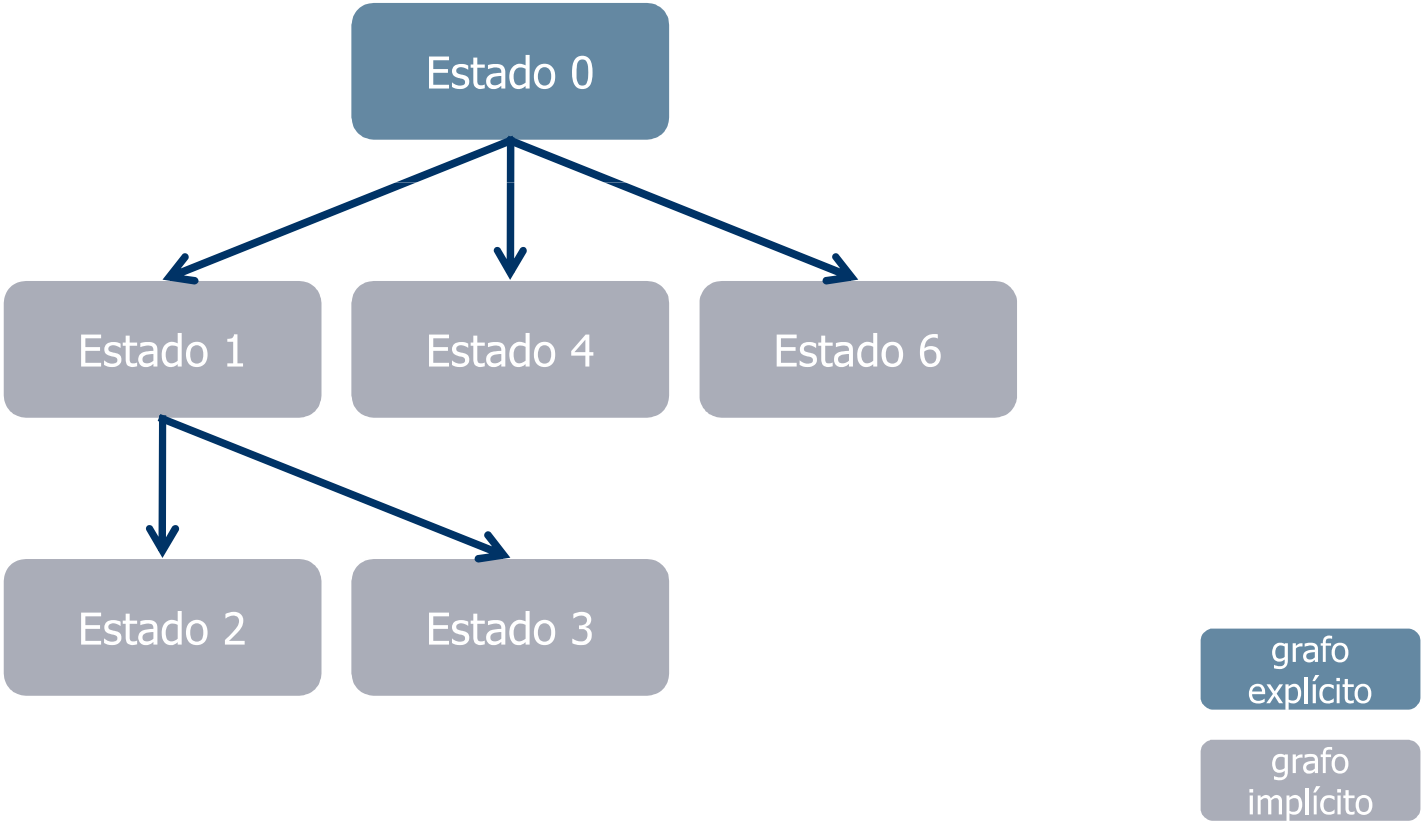
Busca retroactiva

- Estratexias tentativas con información:

  retroactiva (*backtracking*)

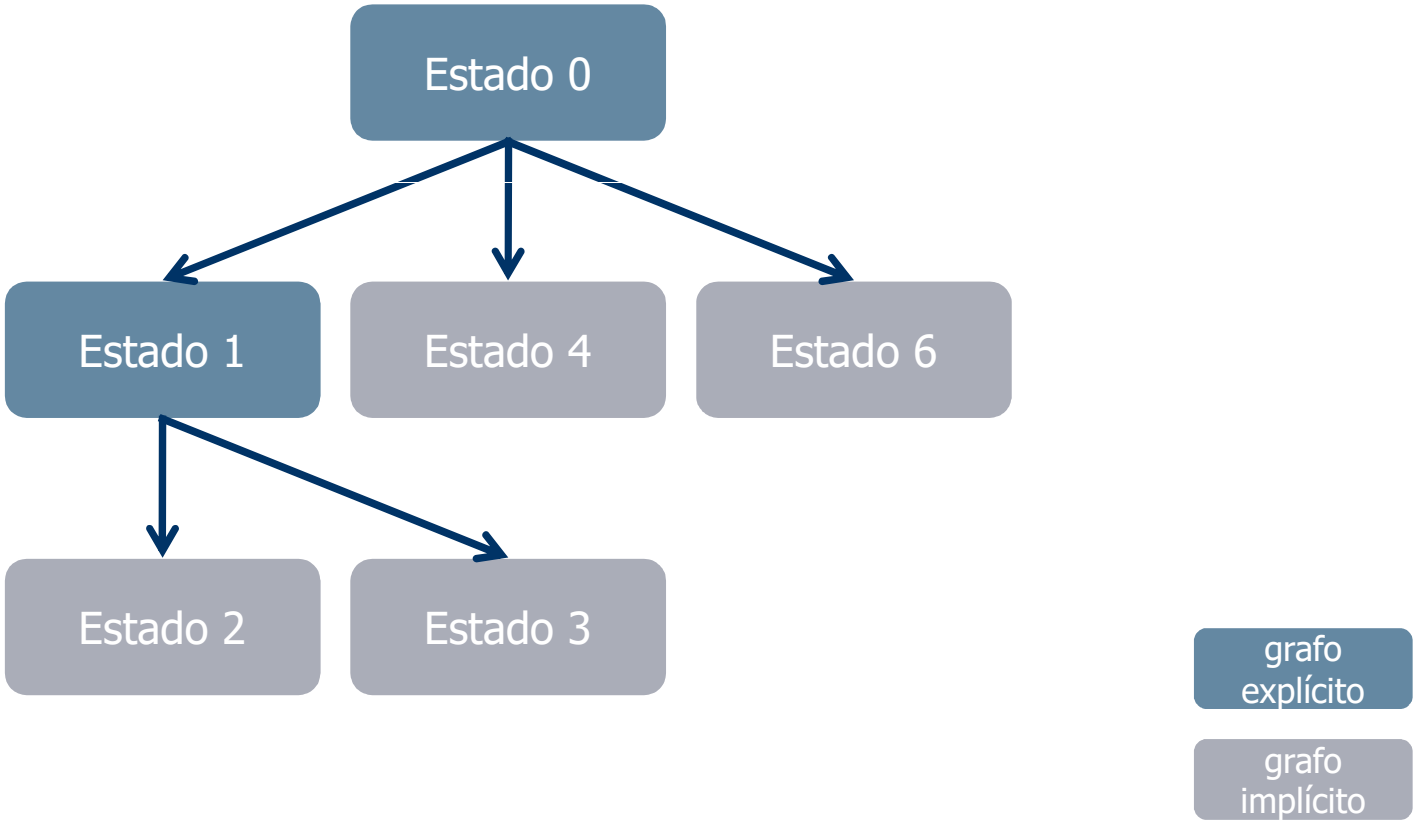
  - Só gardamos un nó fillo de cada estado –mantense o camiño desde o estado inicial ata o estado actual-
  - O grafo explícito é unha lista
  - Parada cando chegamos á meta ou non temos máis operadores que aplicar

Busca retroactiva



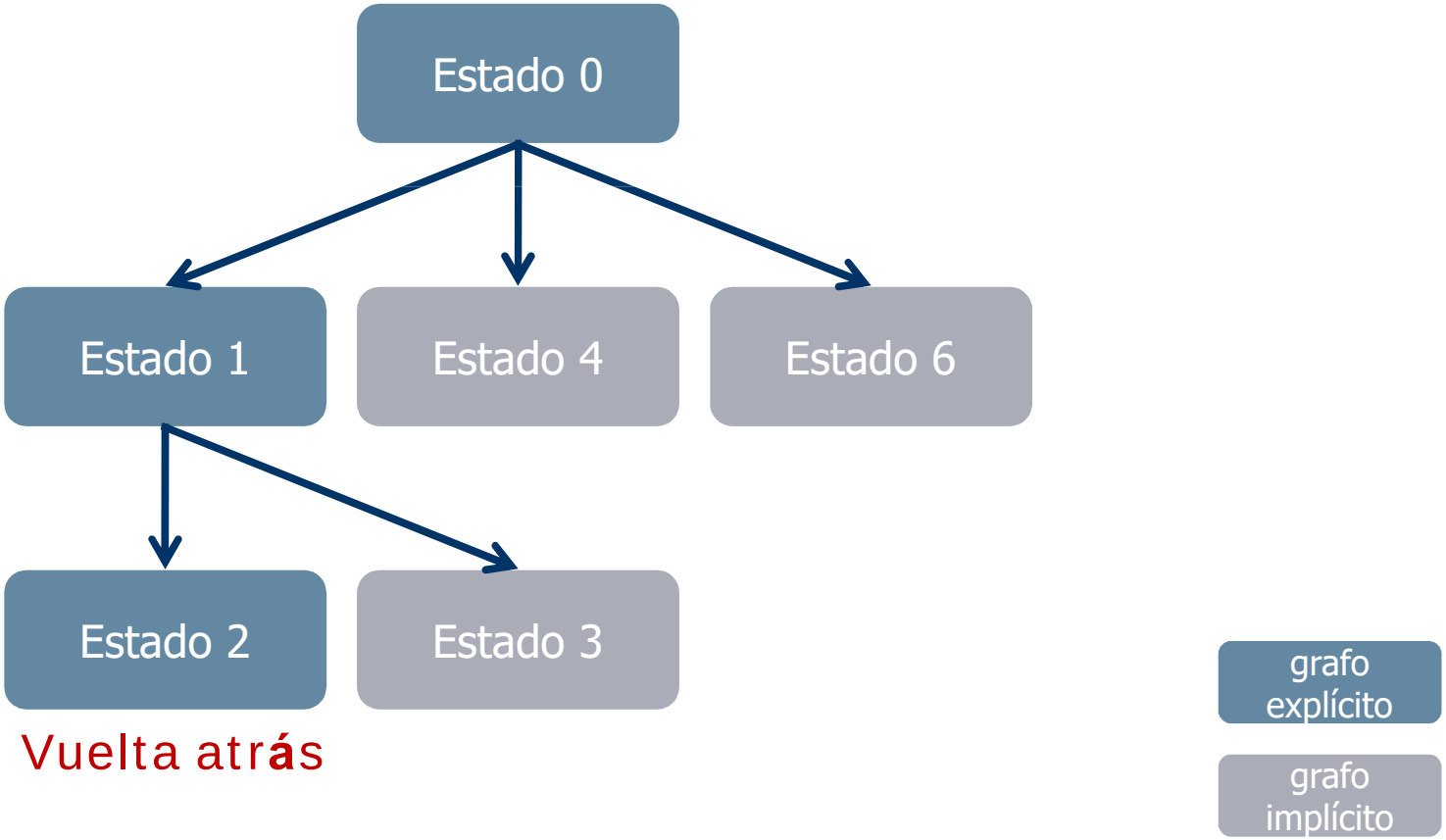
Estratexia tentativa retroactiva

Busca retroactiva



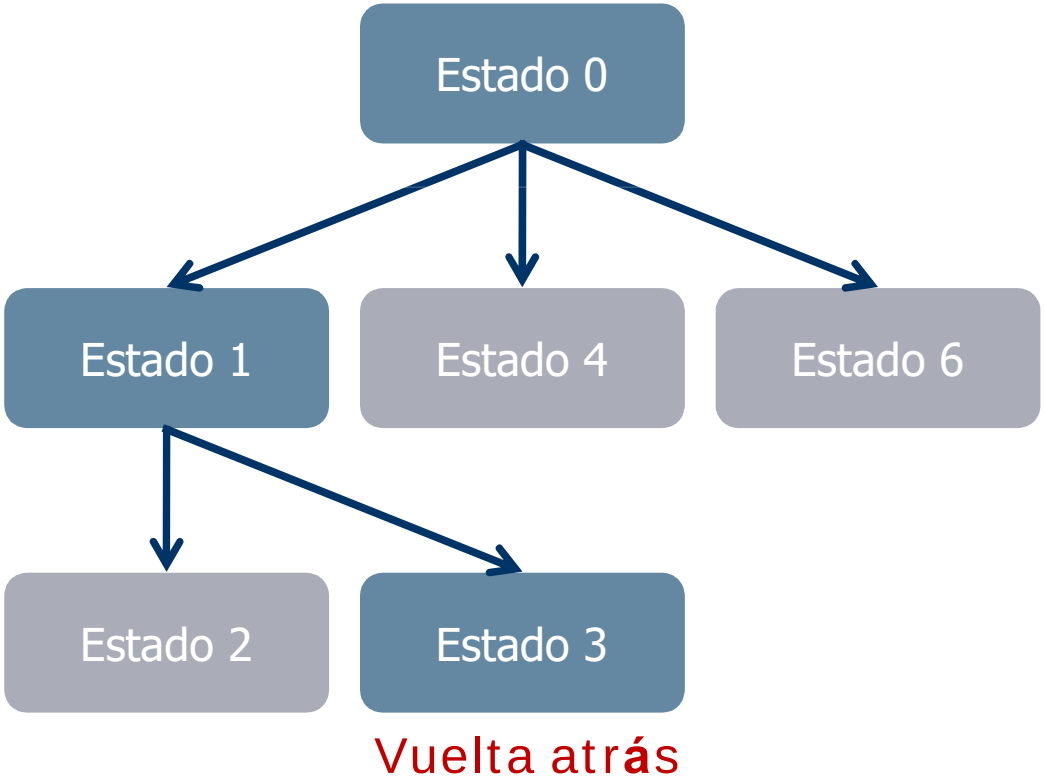
Estratexia tentativa retroactiva

Busca retroactiva



Estratexia tentativa retroactiva

Busca retroactiva

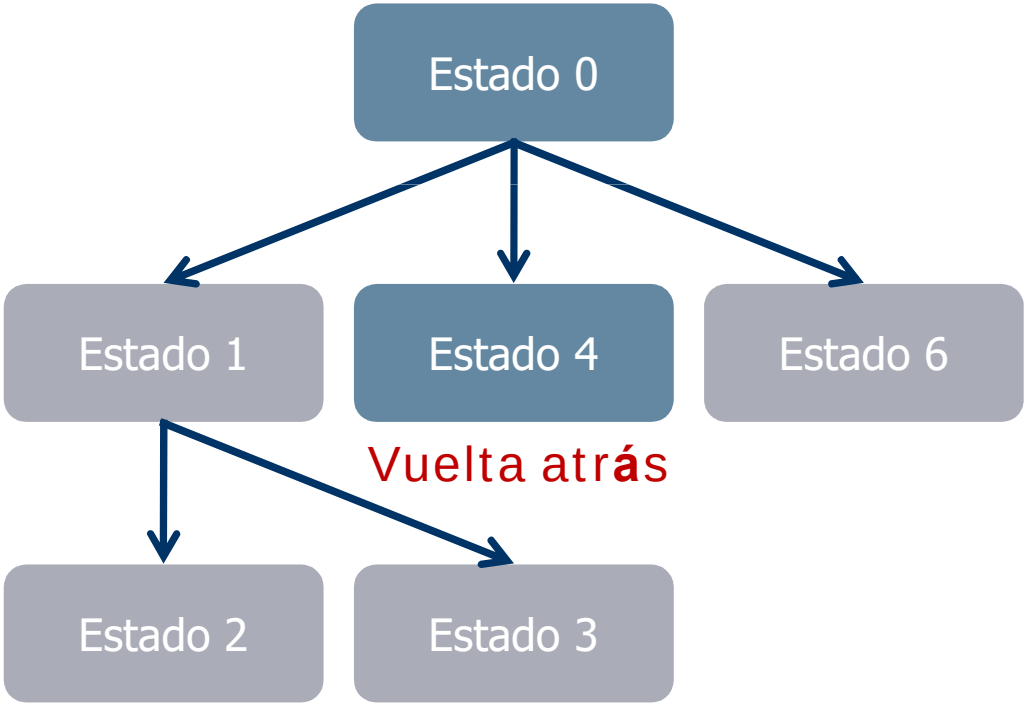


grafo explícito

grafo implícito

Estratexia tentativa retroactiva

Busca retroactiva

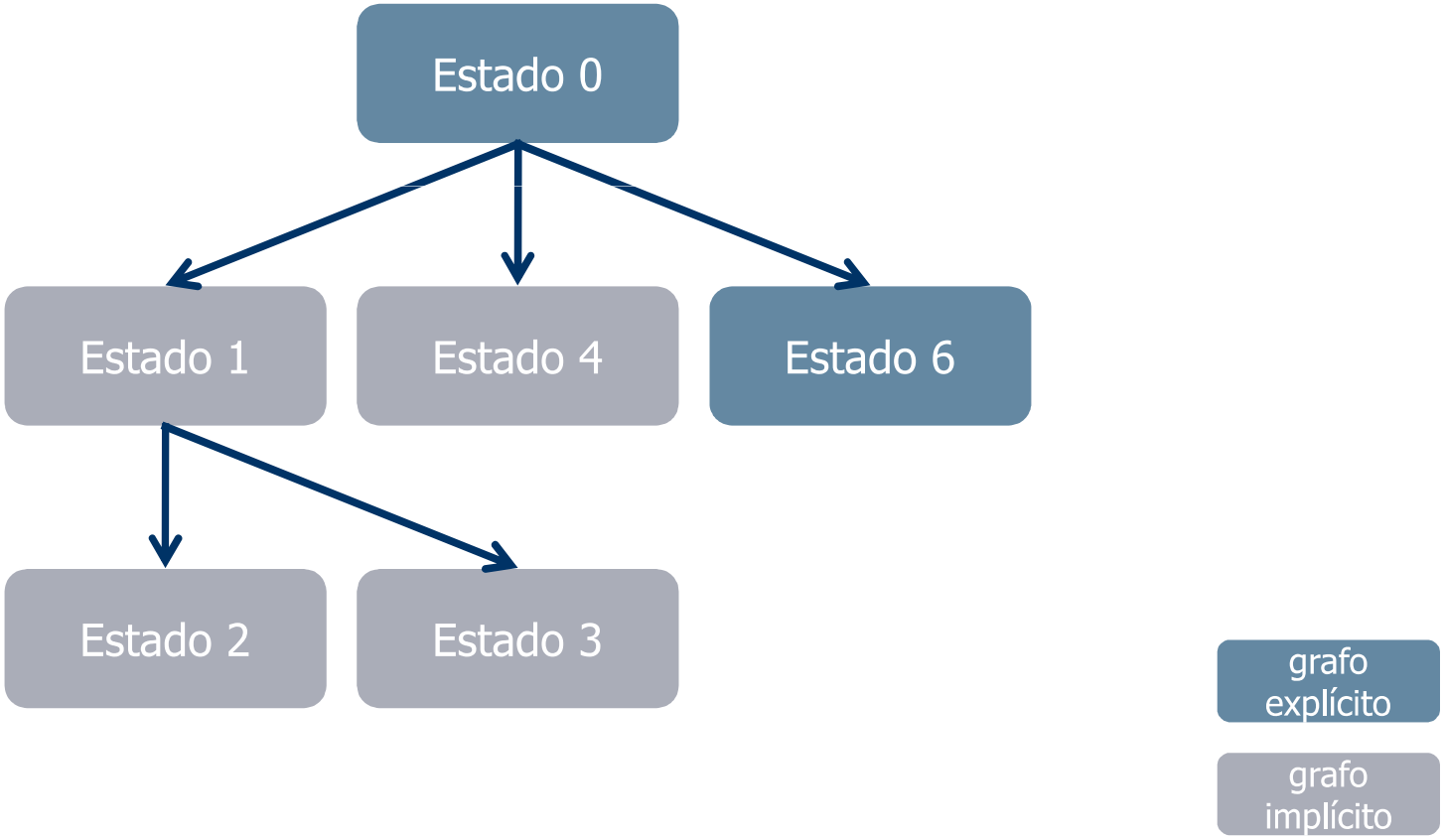


grafo explícito

grafo implícito

Estratexia tentativa retroactiva

Busca retroactiva



Estratexia tentativa retroactiva

Busca heurística: Algoritmo A*

1.- Establecer el nodo s como origen. Hacer f(s)=0, y f(i)=∞ para todos los nodos i diferentes del nodo s. Iniciar el conjunto Q vacío.

2.- Calcular el valor de f(s) y mover el nodo s al conjunto Q .

3.- Seleccionar el nodo i del conjunto Q que presente menor valor de la función f(i) y eliminarlo del conjunto Q.

4.- Analizar los nodos vecinos j de i. Para cada enlace (i, j) con coste cij hacer:

4.1.-Calcular: f(j)'=g(i)+cij +h(j)

4.2.-Si f(j)'<f(j),

4.2.1.-Actualizar la etiqueta de j y su nuevo valor será: f(j)= g(i)+cij +h(j)

4.2.2.-Insertar el nodo j en Q

4.3.-Si f(j)'≥f(j)

4.3.1.-Dejar la etiqueta de j como está, con su valor f(j)

5.- Si Q está vacío el algoritmo se termina. Si no está vacío, volver al paso 3.

Busca heurística

	Primeiro o mellor	**A***
Busca completa	Non	Si
Busca óptima	Non	Si, se h(n) é admisible

Avaliación de estratexias de busca con heurísticas

Como inventar heurísticas?

- Relaxación do problema de partida
  - Exemplo co 8-quebracabezas

  Problema real:

  - A pode pasar a B se A é contigua a B e B está baleira

  Problemas relaxados:

  - A pode pasar a B se A é contigua a B e esta está baleira
  - A pode pasar a B se B está baleira
  - A pode pasar a B en todo caso